

CarGo Rentals

Car Rental Management System

Database Management System | Open-Ended Lab Report

Item	Details
Course	Database Management System (Lab)
Assessment	Open-Ended Lab — CLO-6 (C-4: Analyze), CLO-7 (C-5: Design/Develop)
Student Name	Jawahir Ali
Roll no	2024-SE-34
Instructor	Engr. Muhammad Awais
Submission Date	21-09-2026
Tools Used	XAMPP (Apache + MariaDB), phpMyAdmin
Database Name	<code>cargo_rentals</code>

Overview

CarGo Rentals is a local car rental company that rents vehicles to customers on a daily or weekly basis. The company currently stores customer, vehicle, rental, return and payment information in spreadsheets. As the number of rentals grows this causes duplicate information, inconsistent records and difficulty in generating reports.

This report documents the relational database that was designed and implemented to replace the spreadsheets. The database stores customers and vehicles, records rentals and returns, calculates rental charges, keeps payment information, and prevents a vehicle from being rented while it is already rented. It also provides the management reports requested in the scenario.

Everything was implemented in MariaDB using XAMPP and phpMyAdmin. The complete SQL script ([cargo_rentals.sql](#)) is submitted together with this report. The sections below follow the lab tasks in order; each section contains the SQL used and the screenshots of its execution.

Task 1: Database Design and Implementation

1.1 Tables, keys and constraints

The scenario was analysed to identify the entities (customer, vehicle, vehicle model, rental, payment) and the relationships between them. One customer can make many rentals; one vehicle can appear in many rentals over time; one model can have many vehicles; one rental can have many payments. This gives the following five tables.

Table	Purpose	Primary key	Foreign keys	Main constraints
customers	Customer identity, contact and licence details	customer_id	—	NOT NULL on name, phone, CNIC, licence; UNIQUE on phone, CNIC, licence_no, email; DEFAULT current date for registered_on
vehicle_models	Brand, model name and standard daily rate	model_id	—	UNIQUE (brand, model_name); CHECK daily_rate > 0
vehicles	Individual cars of the fleet	vehicle_id	model_id → vehicle_models	UNIQUE vehicle_number ; CHECK model_year 2000–2035; status ENUM (Available, Rented, Maintenance) DEFAULT 'Available'
rentals	Each rental and its return	rental_id	customer_id → customers; vehicle_id → vehicles	CHECK due_date > rental_date ; CHECK return_date is NULL or ≥ rental_date ; CHECK rate and total > 0; return_date stays NULL while the car is out
payments	Payments made against a rental	payment_id	rental_id → rentals	CHECK amount > 0; method ENUM DEFAULT 'Cash'; DEFAULT current date for payment_date

1.2 Design justification

- **vehicle_models is a separate table.** The daily rate depends on the model (e.g. every Corolla has the same rate), not on the individual car. Storing it once removes repetition and makes a rate change a single-row update.
- **payments is a separate table.** A rental can be paid in more than one payment (advance plus balance). Rental 5 in the sample data has two payments, which a single payment column could not store.
- **daily_rate_applied and total_charge are stored in rentals.** They record the price agreed at booking time, so old rentals keep their original charge even if the model's rate changes later.
- **return_date is NULL while a vehicle is rented.** This makes “currently rented” easy to identify (**return_date IS NULL**) and avoids a separate returns table for a one-to-one relationship.
- **Foreign keys use ON DELETE RESTRICT.** Customers and vehicles that have rental history cannot be deleted by mistake, so historical records stay intact. **ON UPDATE CASCADE** keeps keys consistent.
- **vehicles.status** is kept as a fast availability flag. Because it is derived from the rentals, it is maintained automatically by triggers (Task 5) so that it cannot get out of step with the rental data.

1.3 Sample data

Realistic sample data was inserted to demonstrate every feature: 8 customers, 6 vehicle models, 8 vehicles, 11 rentals (8 completed and 3 active) and 12 payments. The data deliberately includes

two customers who have never rented (for the LEFT JOIN queries), a vehicle that is under maintenance and never rented, a rental paid in two instalments, active rentals with only an advance paid, and one rental from 2025 to make date-range queries meaningful.

1.4 phpMyAdmin Designer view showing all five tables and their foreign-key relationships

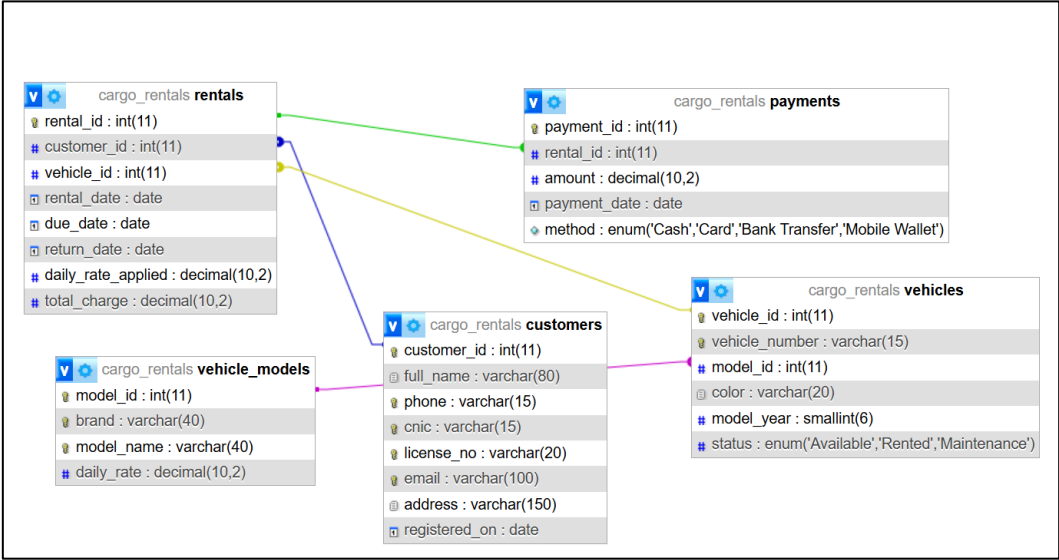


Figure 1.1: phpMyAdmin Designer view showing all five tables and their foreign-key relationships

1.5 Structure tab of each table (customers, vehicle_models, vehicles, rentals, payments)

Table	Action	Rows	Type	Collation	Size
☐ customers	★ Browse Structure Search Insert Empty Drop	8	InnoDB	utf8mb4_general_ci	80.0 KiB
☐ payments	★ Browse Structure Search Insert Empty Drop	12	InnoDB	utf8mb4_general_ci	32.0 KiB
☐ rentals	★ Browse Structure Search Insert Empty Drop	11	InnoDB	utf8mb4_general_ci	48.0 KiB
☐ vehicles	★ Browse Structure Search Insert Empty Drop	8	InnoDB	utf8mb4_general_ci	48.0 KiB
☐ vehicle_models	★ Browse Structure Search Insert Empty Drop	6	InnoDB	utf8mb4_general_ci	32.0 KiB
5 tables	Sum	45	InnoDB	utf8mb4_general_ci	240.0 KiB

Figure 1.2: Structure tab of each table (customers, vehicle_models, vehicles, rentals, payments)

1.6:Customer Table

	customer_id	full_name	phone	cnic	license_no	email	address	registered_on
☐ Edit Copy Delete	1	Ali Khan	0300-1234567	42101-1234567-1	KHI-2019-1001	ali.khan@example.com	Gulshan-e-Iqbal, Karachi	2026-01-10
☐ Edit Copy Delete	2	Sara Ahmed	0321-2345678	35202-2345678-3	LHR-2018-2002	sara.ahmed@example.com	Model Town, Lahore	2026-02-14
☐ Edit Copy Delete	3	Usman Raza	0333-3456789	37405-3456789-5	RWP-2020-3003	usman.raza@example.com	Satellite Town, Rawalpindi	2026-03-05
☐ Edit Copy Delete	4	Ayesha Noor	0345-4567890	61101-4567890-7	ISB-2017-4004	ayesha.noor@example.com	F-10, Islamabad	2026-03-22
☐ Edit Copy Delete	5	Bilal Hussain	0301-5678901	34101-5678901-9	GRW-2021-5005	bilal.hussain@example.com	Gujranwala Cantt	2026-05-01
☐ Edit Copy Delete	6	Hina Malik	0312-6789012	42201-6789012-2	KHI-2016-6006	hina.malik@example.com	DHA Phase 5, Karachi	2026-06-18
☐ Edit Copy Delete	7	Zain Abbas	0322-7890123	35201-7890123-4	LHR-2022-7007	zain.abbas@example.com	Johar Town, Lahore	2026-08-02
☐ Edit Copy Delete	8	Fatima Shah	0334-8901234	71501-8901234-6	SKD-2019-8008	fatima.shah@example.com	Saddar, Rawalpindi	2026-09-01

1.7 Payment Table

	payment_id	rental_id	amount	payment_date	method
☐ Edit Copy Delete	1	1	15000.00	2026-09-01	Cash
☐ Edit Copy Delete	2	2	32500.00	2026-08-10	Card
☐ Edit Copy Delete	3	3	21000.00	2026-08-20	Bank Transfer
☐ Edit Copy Delete	4	4	7000.00	2026-09-08	Cash
☐ Edit Copy Delete	5	5	40000.00	2026-09-05	Card
☐ Edit Copy Delete	6	5	44000.00	2026-09-12	Card
☐ Edit Copy Delete	7	6	6000.00	2026-09-12	Cash
☐ Edit Copy Delete	8	7	35000.00	2026-07-01	Bank Transfer
☐ Edit Copy Delete	9	8	9000.00	2025-12-20	Cash
☐ Edit Copy Delete	10	9	20000.00	2026-09-15	Mobile Wallet
☐ Edit Copy Delete	11	10	10000.00	2026-09-18	Cash
☐ Edit Copy Delete	12	11	15000.00	2026-09-17	Card

1.8 Rental Table

	rental_id	customer_id	vehicle_id	rental_date	due_date	return_date	daily_rate_applied	total_charge
 Copy  Delete	1	1	1	2026-09-01	2026-09-04	2026-09-04	5000.00	15000.00
 Copy  Delete	2	2	2	2026-08-10	2026-08-15	2026-08-15	6500.00	32500.00
 Copy  Delete	3	3	3	2026-08-20	2026-08-27	2026-08-27	3000.00	21000.00
 Copy  Delete	4	1	4	2026-09-08	2026-09-10	2026-09-10	3500.00	7000.00
 Copy  Delete	5	4	5	2026-09-05	2026-09-12	2026-09-12	12000.00	84000.00
 Copy  Delete	6	3	3	2026-09-12	2026-09-14	2026-09-14	3000.00	6000.00
 Copy  Delete	7	1	7	2026-07-01	2026-07-08	2026-07-08	5000.00	35000.00
 Copy  Delete	8	4	3	2025-12-20	2025-12-23	2025-12-23	3000.00	9000.00
 Copy  Delete	9	5	2	2026-09-15	2026-09-22	NULL	6500.00	45500.00
 Copy  Delete	10	2	6	2026-09-18	2026-09-21	NULL	9000.00	27000.00
 Copy  Delete	11	6	1	2026-09-17	2026-09-24	NULL	5000.00	35000.00

1.9 Vehicle Table

	vehicle_id	vehicle_number	model_id	color	model_year	status
 Delete	1	ABC-123	1	White	2022	Rented
 Delete	2	LEA-4521	2	Black	2021	Rented
 Delete	3	KHI-7788	3	Silver	2023	Available
 Delete	4	ISB-3344	4	Red	2020	Available
 Delete	5	LHR-9012	5	Grey	2022	Available
 Delete	6	GLT-2210	6	Blue	2023	Rented
 Delete	7	ABD-456	1	Silver	2019	Available
 Delete	8	RWP-6677	2	White	2018	Maintenance

1.10 Vehicle_models

	model_id	brand	model_name	daily_rate
 Edit  Copy  Delete	1	Toyota	Corolla	5000.00
 Edit  Copy  Delete	2	Honda	Civic	6500.00
 Edit  Copy  Delete	3	Suzuki	Alto	3000.00
 Edit  Copy  Delete	4	Suzuki	Cultus	3500.00
 Edit  Copy  Delete	5	Toyota	Fortuner	12000.00
 Edit  Copy  Delete	6	Kia	Sportage	9000.00

Task 2: Normalization

2.1 The original (unnormalized) record

The company currently stores every rental as one wide record:

RentalID	Customer Name	Customer Phone	Vehicle Number	Vehicle Model	Daily Rate	Rental Date	Return Date	Payment Amount
R001	Ali Khan	0300-1234567	ABC-123	Toyota Corolla	5000	2026-09-01	2026-09-04	15000

Problems with this structure:

- **Redundancy:** the customer's name and phone are repeated on every rental of that customer; the vehicle model and daily rate are repeated on every rental of that vehicle.
- **Inconsistency:** the same customer could be typed as "Ali Khan" in one row and "A. Khan" in another, and the same model could carry two different daily rates.
- **Payments:** one **PaymentAmount** column cannot represent an advance payment followed by a final payment.

2.2 First Normal Form (1NF)

Rule: Every attribute must hold a single (atomic) value, there must be no repeating groups, and the relation must have a primary key.

Problem found	Why it breaks 1NF	Fix applied
VehicleModel = "Toyota Corolla"	Two facts (brand and model) in one value; brand cannot be searched or sorted on its own	Split into Brand and Model
PaymentAmount for a rental paid in instalments	Needs several values per rental (a repeating group such as Payment1, Payment2, ...)	Move payments to their own relation, one row per payment
No stable identifier	Rows could not be uniquely identified reliably	RentalID is the primary key of Rental

Relations in 1NF:

Rental (RentalID PK, CustomerName, CustomerPhone, VehicleNumber, Brand, Model, DailyRate, RentalDate, ReturnDate)

Payment(PaymentID PK, RentalID FK, PaymentAmount, PaymentDate, Method)

Anomalies resolved at this stage: Multi-valued and repeating payment data no longer forces empty or duplicated columns; each fact (brand, model) can be queried on its own.

2.3 Second Normal Form (2NF)

Rule: The relation is in 1NF and no non-key attribute depends on only part of a composite primary key.

Both **Rental** (key **RentalID**) and **Payment** (key **PaymentID**) have a single-attribute key. A partial dependency is only possible with a composite key, so every non-key attribute depends on the whole key and both relations are already in 2NF.

Extended case. If the business allowed one rental to contain several vehicles, the key of **Rental** would become (**RentalID**, **VehicleNumber**), and partial dependencies would appear:

```
RentalID      -> CustomerName, CustomerPhone, RentalDate, ReturnDate
(part of key)
VehicleNumber -> Brand, Model, DailyRate
(part of key)
```

2NF decomposition:

```
Rental      (RentalID PK, CustomerName, CustomerPhone, RentalDate,
ReturnDate)
Vehicle      (VehicleNumber PK, Brand, Model, DailyRate)
RentalVehicle (RentalID FK, VehicleNumber FK)
```

In the implemented database each rental is for one vehicle, as the scenario describes, so **rentals** stores the vehicle directly. The junction table above can be added later without changing any other table.

Anomalies resolved at this stage: vehicle details are no longer repeated for every rental line, and customer details are no longer tied to a rental–vehicle combination.

2.4 Third Normal Form (3NF)

Rule: The relation is in 2NF and no non-key attribute depends on another non-key attribute (no transitive dependency).

Transitive dependencies found in **Rental**:

```
RentalID -> CustomerPhone -> CustomerName
RentalID -> VehicleNumber -> Model -> DailyRate
```

The customer's name is determined by the customer, not by the rental, and the daily rate is determined by the vehicle model, not by the rental. Each determinant is moved into its own relation with a surrogate key. The resulting 3NF design is exactly the implemented schema:

Relation	Attributes (PK = primary key, FK = foreign key)
customers	customer_id (PK), full_name, phone, cnic, license_no, email, address, registered_on
vehicle_models	model_id (PK), brand, model_name, daily_rate
vehicles	vehicle_id (PK), vehicle_number, model_id (FK), color, model_year, status

Relation	Attributes (PK = primary key, FK = foreign key)
rentals	rental_id (PK), customer_id (FK), vehicle_id (FK), rental_date, due_date, return_date, daily_rate_applied, total_charge
payments	payment_id (PK), rental_id (FK), amount, payment_date, method

How the original record R001 is now stored, without repetition:

Table	Row
customers	1, Ali Khan, 0300-1234567, ...
vehicle_models	1, Toyota, Corolla, 5000.00
vehicles	1, ABC-123, model_id = 1, White, 2022, ...
rentals	1, customer_id = 1, vehicle_id = 1, 2026-09-01, due 2026-09-04, returned 2026-09-04, rate 5000, total 15000
payments	1, rental_id = 1, 15000.00, 2026-09-01, Cash

2.5 Anomalies removed by normalization

Anomaly	Problem in the single-record structure	How the 3NF design fixes it
Insertion	A new customer or a new vehicle cannot be stored until a rental exists for it	Customers, models and vehicles are stored independently of rentals
Update	Changing the Corolla's daily rate or a customer's phone number means updating every rental row; missing one causes inconsistency	The rate is stored once in vehicle_models and the phone once in customers
Deletion	Deleting a customer's only rental also deletes the customer and vehicle information	Deleting a rental never removes customer or vehicle data (and foreign keys block accidental deletion)
Redundancy	Name, phone, model and rate repeated in every rental	Each fact is stored exactly once and referenced by key

Task 3 : - JOIN Queries

Query 1: Every rental with customer, vehicle and dates

Business question: Display the customer's name, vehicle number, vehicle model, rental date and return date for every rental.

Join used: **INNER JOIN** across **rentals**, **customers**, **vehicles** and **vehicle_models**, because only rows that have a match in all four tables are required. Rentals that are still active show NULL in **return_date**.

```
SELECT c.full_name AS customer_name,
       v.vehicle_number,
       CONCAT(m.brand, ' ', m.model_name) AS vehicle_model,
       r.rental_date,
       r.return_date
FROM rentals r
INNER JOIN customers c ON c.customer_id = r.customer_id
INNER JOIN vehicles v ON v.vehicle_id = r.vehicle_id
INNER JOIN vehicle_models m ON m.model_id = v.model_id
ORDER BY r.rental_date;
```

After Running this Query: Output

customer_name	vehicle_number	vehicle_model	rental_date	1	return_date
Ayesha Noor	KHI-7788	Suzuki Alto	2025-12-20		2025-12-23
Ali Khan	ABD-456	Toyota Corolla	2026-07-01		2026-07-08
Sara Ahmed	LEA-4521	Honda Civic	2026-08-10		2026-08-15
Usman Raza	KHI-7788	Suzuki Alto	2026-08-20		2026-08-27
Ali Khan	ABC-123	Toyota Corolla	2026-09-01		2026-09-04
Ayesha Noor	LHR-9012	Toyota Fortuner	2026-09-05		2026-09-12
Ali Khan	ISB-3344	Suzuki Cultus	2026-09-08		2026-09-10
Usman Raza	KHI-7788	Suzuki Alto	2026-09-12		2026-09-14
Bilal Hussain	LEA-4521	Honda Civic	2026-09-15		NULL
Hina Malik	ABC-123	Toyota Corolla	2026-09-17		NULL
Sara Ahmed	GLT-2210	Kia Sportage	2026-09-18		NULL

Figure 3.1: Output of Query 1 (SQL text and result: 11 rentals)

Query 2 — All customers and their rented vehicles

Business question: Display all customers and the vehicles they have rented; customers who never rented must also appear.

Join used: **LEFT JOIN** starting from **customers**, so every customer is kept. For customers without rentals the vehicle columns are NULL (Zain Abbas and Fatima Shah in the sample data).

```

SELECT c.customer_id,
       c.full_name           AS customer_name,
       v.vehicle_number,
       CONCAT(m.brand, ' ', m.model_name) AS vehicle_model,
       r.rental_date
FROM customers c
LEFT JOIN rentals      r ON r.customer_id = c.customer_id
LEFT JOIN vehicles     v ON v.vehicle_id  = r.vehicle_id
LEFT JOIN vehicle_models m ON m.model_id   = v.model_id
ORDER BY c.customer_id, r.rental_date;

```

Output: -

customer_id	customer_name	vehicle_number	vehicle_model	rental_date
1	Ali Khan	ABD-456	Toyota Corolla	2026-07-01
1	Ali Khan	ABC-123	Toyota Corolla	2026-09-01
1	Ali Khan	ISB-3344	Suzuki Cultus	2026-09-08
2	Sara Ahmed	LEA-4521	Honda Civic	2026-08-10
2	Sara Ahmed	GLT-2210	Kia Sportage	2026-09-18
3	Usman Raza	KHI-7788	Suzuki Alto	2026-08-20
3	Usman Raza	KHI-7788	Suzuki Alto	2026-09-12
4	Ayesha Noor	KHI-7788	Suzuki Alto	2025-12-20
4	Ayesha Noor	LHR-9012	Toyota Fortuner	2026-09-05
5	Bilal Hussain	LEA-4521	Honda Civic	2026-09-15
6	Hina Malik	ABC-123	Toyota Corolla	2026-09-17
7	Zain Abbas	NULL	NULL	NULL
8	Fatima Shah	NULL	NULL	NULL

Figure 3.2: customers with no rentals show NULL vehicle columns

Query 3 : - All vehicles with their current rental

Business question: Display all vehicles and their current rental information; vehicles that are not rented now must also appear.

Join used: **LEFT JOIN** starting from **vehicles**. The condition **r.return_date IS NULL** is placed in the **ON** clause, not in **WHERE**. If it were in **WHERE**, available vehicles (which have no matching active rental) would be filtered out and disappear from the result.

```

SELECT v.vehicle_number,
       CONCAT(m.brand, ' ', m.model_name) AS vehicle_model,
       v.status,
       c.full_name           AS rented_by,
       r.rental_date,
       r.due_date
FROM vehicles v
INNER JOIN vehicle_models m ON m.model_id = v.model_id
LEFT JOIN rentals      r ON r.vehicle_id = v.vehicle_id AND r.return_date IS NULL
LEFT JOIN customers    c ON c.customer_id = r.customer_id
ORDER BY v.vehicle_id;

```

Output: -

vehicle_number	vehicle_model	status	rented_by	rental_date	due_date
ABC-123	Toyota Corolla	Rented	Hina Malik	2026-09-17	2026-09-24
LEA-4521	Honda Civic	Rented	Bilal Hussain	2026-09-15	2026-09-22
KHI-7788	Suzuki Alto	Available	NULL	NULL	NULL
ISB-3344	Suzuki Cultus	Available	NULL	NULL	NULL
LHR-9012	Toyota Fortuner	Available	NULL	NULL	NULL
GLT-2210	Kia Sportage	Rented	Sara Ahmed	2026-09-18	2026-09-21
ABD-456	Toyota Corolla	Available	NULL	NULL	NULL
RWP-6677	Honda Civic	Maintenance	NULL	NULL	NULL

Figure 3.3: All 8 vehicles; 3 have an active renter

Query 4:- Total rentals per customer

Business question: Display the total number of rentals made by each customer, including customers with no rentals.

Join used: `LEFT JOIN` with `GROUP BY`. `COUNT(r.rental_id)` counts only real rentals; `COUNT(*)` would wrongly report 1 for customers with no rentals because the outer join still produces one row for them.

```
SELECT c.customer_id,
       c.full_name      AS customer_name,
       COUNT(r.rental_id) AS total_rentals
FROM customers c
LEFT JOIN rentals r ON r.customer_id = c.customer_id
GROUP BY c.customer_id, c.full_name
ORDER BY total_rentals DESC, c.customer_id;
```

Output: -

customer_id	customer_name	total_rentals
1	Ali Khan	3
2	Sara Ahmed	2
3	Usman Raza	2
4	Ayesha Noor	2
5	Bilal Hussain	1
6	Hina Malik	1
7	Zain Abbas	0
8	Fatima Shah	0

Task 4: - View

The view `vw_rental_report` combines customer, vehicle, rental and payment information into one consolidated rental report. It calculates the rental days, the total amount paid (summed over all payments of the rental), the balance still due, the rental status (Active / Completed) and the payment status (Paid / Partially Paid / Unpaid).

```

1 CREATE OR REPLACE VIEW vw_rental_report AS
2 SELECT r.rental_id,
3        c.full_name           AS customer_name,
4        c.phone              AS customer_phone,
5        v.vehicle_number,
6        CONCAT(m.brand, ' ', m.model_name) AS vehicle_model,
7        r.rental_date, r.due_date, r.return_date,
8        DATEDIFF(r.due_date, r.rental_date) AS rental_days,
9        r.daily_rate_applied, r.total_charge,
10       COALESCE(p.total_paid, 0) AS amount_paid,
11       r.total_charge - COALESCE(p.total_paid, 0) AS balance_due,
12       CASE WHEN r.return_date IS NULL THEN 'Active' ELSE 'Completed' END AS rental_status,
13       CASE WHEN COALESCE(p.total_paid, 0) >= r.total_charge THEN 'Paid'
14            WHEN COALESCE(p.total_paid, 0) = 0 THEN 'Unpaid'
15            ELSE 'Partially Paid' END AS payment_status
16 FROM rentals r
17 INNER JOIN customers c ON c.customer_id = r.customer_id
18 INNER JOIN vehicles v ON v.vehicle_id = r.vehicle_id
19 INNER JOIN vehicle_models m ON m.model_id = v.model_id
20 LEFT JOIN (SELECT rental_id, SUM(amount) AS total_paid
21            FROM payments GROUP BY rental_id) p ON p.rental_id = r.rental_id;
22

```

Why the view is useful

The view gives management a complete rental report with a single, simple query, so nobody has to rewrite the four-table join and the payment totals every time. It shows at a glance which vehicles are currently rented and which customers still owe money, which supports payment follow-up and reporting. Because the logic is stored in one place, every report uses the same definitions of balance and status.

Demonstration

```
SELECT * FROM vw_rental_report ORDER BY rental_id;
```

```
SELECT * FROM vw_rental_report
WHERE rental_status = 'Active' AND balance_due > 0;
```

Successful creation of the view vw_rental_report

✓ MySQL returned an empty result set (i.e. zero rows). (Query took 0.0009 seconds.)

```
CREATE OR REPLACE VIEW vw_rental_report AS SELECT r.rental_id, c.full_name AS customer_name, c.phone AS customer_phone,
v.vehicle_number, CONCAT(m.brand, ' ', m.model_name) AS vehicle_model, r.rental_date, r.due_date, r.return_date, DATEDIFF(r.due_date,
r.rental_date) AS rental_days, r.daily_rate_applied, r.total_charge, COALESCE(p.total_paid, 0) AS amount_paid, r.total_charge -
COALESCE(p.total_paid, 0) AS balance_due, CASE WHEN r.return_date IS NULL THEN 'Active' ELSE 'Completed' END AS rental_status, CASE
WHEN COALESCE(p.total_paid, 0) >= r.total_charge THEN 'Paid' WHEN COALESCE(p.total_paid, 0) = 0 THEN 'Unpaid' ELSE 'Partially Paid' END
AS payment_status FROM rentals r INNER JOIN customers c ON c.customer_id = r.customer_id INNER JOIN vehicles v ON v.vehicle_id =
r.vehicle_id
```

Figure 4.1: Successful creation of the view vw_rental_report

All rentals with paid amount and balance

rental_id	customer_name	customer_phone	vehicle_number	vehicle_model	rental_date	due_date	return_date	rental_days
1	Ali Khan	0300-1234567	ABC-123	Toyota Corolla	2026-09-01	2026-09-04	2026-09-04	3
2	Sara Ahmed	0321-2345678	LEA-4521	Honda Civic	2026-08-10	2026-08-15	2026-08-15	5
3	Usman Raza	0333-3456789	KHI-7788	Suzuki Alto	2026-08-20	2026-08-27	2026-08-27	7
4	Ali Khan	0300-1234567	ISB-3344	Suzuki Cultus	2026-09-08	2026-09-10	2026-09-10	2
5	Ayesha Noor	0345-4567890	LHR-9012	Toyota Fortuner	2026-09-05	2026-09-12	2026-09-12	7
6	Usman Raza	0333-3456789	KHI-7788	Suzuki Alto	2026-09-12	2026-09-14	2026-09-14	2
7	Ali Khan	0300-1234567	ABD-456	Toyota Corolla	2026-07-01	2026-07-08	2026-07-08	7
8	Ayesha Noor	0345-4567890	KHI-7788	Suzuki Alto	2025-12-20	2025-12-23	2025-12-23	3
9	Bilal Hussain	0301-5678901	LEA-4521	Honda Civic	2026-09-15	2026-09-22	NULL	7
10	Sara Ahmed	0321-2345678	GLT-2210	Kia Sportage	2026-09-18	2026-09-21	NULL	3
11	Hina Malik	0312-6789012	ABC-123	Toyota Corolla	2026-09-17	2026-09-24	NULL	7

daily_rate_applied	total_charge	amount_paid	balance_due	rental_status	payment_status
5000.00	15000.00	15000.00	0.00	Completed	Paid
6500.00	32500.00	32500.00	0.00	Completed	Paid
3000.00	21000.00	21000.00	0.00	Completed	Paid
3500.00	7000.00	7000.00	0.00	Completed	Paid
12000.00	84000.00	84000.00	0.00	Completed	Paid
3000.00	6000.00	6000.00	0.00	Completed	Paid
5000.00	35000.00	35000.00	0.00	Completed	Paid
3000.00	9000.00	9000.00	0.00	Completed	Paid
6500.00	45500.00	20000.00	25500.00	Active	Partially Paid
9000.00	27000.00	10000.00	17000.00	Active	Partially Paid
5000.00	35000.00	15000.00	20000.00	Active	Partially Paid

Figure 4.2: All rentals with paid amount and balance

Active rentals with a pending balance

rental_id	customer_name	customer_phone	vehicle_number	rental_date	due_date	total_charge	amount_paid	balance_due	rental_status
9	Bilal Hussain	0301-5678901	LEA-4521	2026-09-15	2026-09-22	45500.00	20000.00	25500.00	Active
10	Sara Ahmed	0321-2345678	GLT-2210	2026-09-18	2026-09-21	27000.00	10000.00	17000.00	Active
11	Hina Malik	0312-6789012	ABC-123	2026-09-17	2026-09-24	35000.00	15000.00	20000.00	Active

Figure 4.3: Active rentals with a pending balance

Task 5: - Trigger

Business rule: a vehicle must not be available for another rental while it is already rented. This rule is enforced inside the database by three triggers, so it holds no matter which application or user inserts the data.

Trigger	Event	Action
trg_prevent_double_rental	BEFORE INSERT on <code>rentals</code>	Rejects a new active rental with an error message if the vehicle status is not 'Available' (already rented or under maintenance)
trg_mark_vehicle_rented	AFTER INSERT on <code>rentals</code>	Sets the vehicle status to 'Rented' after a successful active rental
trg_mark_vehicle_available	AFTER UPDATE on <code>rentals</code>	When <code>return_date</code> is filled in, sets the vehicle status back to 'Available'

SQL

```
CREATE TRIGGER trg_prevent_double_rental
BEFORE INSERT ON rentals
FOR EACH ROW
BEGIN
    DECLARE v_status VARCHAR(20);
    IF NEW.return_date IS NULL THEN
        SELECT status INTO v_status FROM vehicles WHERE vehicle_id = NEW.vehicle_id;
        IF v_status <> 'Available' THEN
            SIGNAL SQLSTATE '45000'
            SET MESSAGE_TEXT = 'Rental rejected: vehicle is not available (already rented or under maintenance).';
        END IF;
    END IF;
END$$

CREATE TRIGGER trg_mark_vehicle_rented
AFTER INSERT ON rentals
FOR EACH ROW
BEGIN
    IF NEW.return_date IS NULL THEN
        UPDATE vehicles SET status = 'Rented' WHERE vehicle_id = NEW.vehicle_id;
    END IF;
END$$
```

```

CREATE TRIGGER trg_mark_vehicle_available
AFTER UPDATE ON rentals
FOR EACH ROW
BEGIN
    IF OLD.return_date IS NULL AND NEW.return_date IS NOT NULL THEN
        UPDATE vehicles SET status = 'Available'
        WHERE vehicle_id = NEW.vehicle_id AND status = 'Rented';
    END IF;
END$$
DELIMITER ;

```

Successful Creation of Three Triggers

✓ MySQL returned an empty result set (i.e. zero rows). (Query took 0.0113 seconds.)

```

CREATE TRIGGER trg_prevent_double_rental BEFORE INSERT ON rentals FOR EACH ROW BEGIN DECLARE v_status VARCHAR(20); IF NEW.return_date
IS NULL THEN SELECT status INTO v_status FROM vehicles WHERE vehicle_id = NEW.vehicle_id; IF v_status <> 'Available' THEN SIGNAL
SQLSTATE '45000' SET MESSAGE_TEXT = 'Rental rejected: vehicle is not available (already rented or under maintenance).'; END IF; END IF;
END;

```

[Edit inline] [Edit] [Create PHP code]

✓ MySQL returned an empty result set (i.e. zero rows). (Query took 0.0144 seconds.)

```

CREATE TRIGGER trg_mark_vehicle_rented AFTER INSERT ON rentals FOR EACH ROW BEGIN IF NEW.return_date IS NULL THEN UPDATE vehicles SET
status = 'Rented' WHERE vehicle_id = NEW.vehicle_id; END IF; END;

```

[Edit inline] [Edit] [Create PHP code]

✓ MySQL returned an empty result set (i.e. zero rows). (Query took 0.0127 seconds.)

```

CREATE TRIGGER trg_mark_vehicle_available AFTER UPDATE ON rentals FOR EACH ROW BEGIN IF OLD.return_date IS NULL AND NEW.return_date IS
NOT NULL THEN UPDATE vehicles SET status = 'Available' WHERE vehicle_id = NEW.vehicle_id AND status = 'Rented'; END IF; END;

```

[Edit inline] [Edit] [Create PHP code]

Figure 5.1: Successful creation of the three triggers

Testing

Test 1 tries to rent vehicle 1 (ABC-123), which is currently rented, and Test 2 tries to rent vehicle 8 (RWP-6677), which is under maintenance. Both must fail with the trigger's error message. The successful rental and the return of a vehicle (which change the status through triggers 2 and 3) are demonstrated with the stored procedure in Task 6.

Test 1 : vehicle already rented must FAIL

```

1 -- Test 1: vehicle already rented
2 INSERT INTO rentals (customer_id, vehicle_id, rental_date, due_date,
3                     daily_rate_applied, total_charge)
4 VALUES (7, 1, '2026-09-20', '2026-09-22', 5000.00, 10000.00);
5

```

Result:

```

Error
SQL query: Copy

-- Test 1: vehicle already rented -> must FAIL
INSERT INTO rentals (customer_id, vehicle_id, rental_date, due_date,
                    daily_rate_applied, total_charge)
VALUES (7, 1, '2026-09-20', '2026-09-22', 5000.00, 10000.00);

MySQL said: ⓘ

#1644 - Rental rejected: vehicle is not available
                    (already rented or under maintenance).

```

Figure 5.2: Error message for the already-rented vehicle

Test 2: vehicle under maintenance must FAIL

```

-- Test 2: vehicle under maintenance
INSERT INTO rentals (customer_id, vehicle_id, rental_date, due_date,
                    daily_rate_applied, total_charge)
VALUES (7, 8, '2026-09-20', '2026-09-22', 6500.00, 13000.00);

```

Result:

```

Error
SQL query: Copy

-- Test 2: vehicle under maintenance
INSERT INTO rentals (customer_id, vehicle_id, rental_date, due_date,
                    daily_rate_applied, total_charge)
VALUES (7, 8, '2026-09-20', '2026-09-22', 6500.00, 13000.00);

MySQL said: ⓘ

#1644 - Rental rejected: vehicle is not available
                    (already rented or under maintenance).

```

Figure 5.3: Error message for the vehicle under maintenance

Statuses unchanged after failed inserts

```

1 SELECT vehicle_number, status
2 FROM vehicles
3 WHERE status = 'Available';

```

Output:

				vehicle_number	status			
☐		Edit		Copy		Delete	KHI-7788	Available
☐		Edit		Copy		Delete	ISB-3344	Available
☐		Edit		Copy		Delete	LHR-9012	Available
☐		Edit		Copy		Delete	ABD-456	Available

Figure 5.4: `SELECT vehicle_number, status FROM vehicles` (statuses unchanged after failed inserts)

Task 6: Stored Procedure

The procedure `sp_register_rental` registers a new rental and calculates the rental charge from the rental period and the vehicle's daily rate, as suggested in the lab manual. It also records an optional advance payment.

Input parameters

Parameter	Type	Meaning
<code>p_customer_id</code>	INT	Customer making the rental
<code>p_vehicle_id</code>	INT	Vehicle to be rented
<code>p_rental_date</code>	DATE	Start date of the rental
<code>p_days</code>	INT	Number of rental days
<code>p_advance</code>	DECIMAL(10,2)	Advance payment (0 if none)
<code>p_method</code>	VARCHAR(20)	Payment method (Cash, Card, Bank Transfer, Mobile Wallet)

Processing logic

- **Validation:** days must be at least 1; the customer must exist; the vehicle must exist; the advance must be between 0 and the total charge. Invalid input raises a clear error message.
- **Calculation:** the vehicle's daily rate is read through `vehicles` and `vehicle_models`;
- `due date = rental date + days`; `total charge = daily rate × days`.
- **Database work:** Inserts the rental (the BEFORE INSERT trigger blocks unavailable vehicles and the AFTER INSERT trigger marks the vehicle as Rented) and inserts the advance payment.
- **Transaction:** Everything runs inside one transaction with an exit handler that rolls back and re-raises any error, so a failed rental never leaves a half-saved payment.
- **Output:** Returns a summary row with the rental id, due date, total charge, advance paid and balance due.

```

DELIMITER $$
|
CREATE PROCEDURE sp_register_rental(
    IN p_customer_id INT, IN p_vehicle_id INT, IN p_rental_date DATE,
    IN p_days INT, IN p_advance DECIMAL(10,2), IN p_method VARCHAR(20))
BEGIN
    DECLARE v_rate DECIMAL(10,2);
    DECLARE v_total DECIMAL(10,2);
    DECLARE v_due DATE;
    DECLARE v_rental_id INT;

    DECLARE EXIT HANDLER FOR SQLEXCEPTION
    BEGIN
        ROLLBACK;
        RESIGNAL;
    END;

    SET p_advance = IFNULL(p_advance, 0);

    IF p_days IS NULL OR p_days < 1 THEN
        SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Rental period must be at least 1 day.';
    END IF;
    IF NOT EXISTS (SELECT 1 FROM customers WHERE customer_id = p_customer_id) THEN
        SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Customer does not exist.';
    END IF;

    SELECT m.daily_rate INTO v_rate
    FROM vehicles v INNER JOIN vehicle_models m ON m.model_id = v.model_id
    WHERE v.vehicle_id = p_vehicle_id;
    IF v_rate IS NULL THEN
        SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Vehicle does not exist.';
    END IF;

    SET v_due = DATE_ADD(p_rental_date, INTERVAL p_days DAY);
    SET v_total = v_rate * p_days;
    IF p_advance < 0 OR p_advance > v_total THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Advance payment must be between 0 and the total charge.';
    END IF;

    START TRANSACTION;
    INSERT INTO rentals (customer_id, vehicle_id, rental_date, due_date,
                        daily_rate_applied, total_charge)
    VALUES (p_customer_id, p_vehicle_id, p_rental_date, v_due, v_rate, v_total);
    SET v_rental_id = LAST_INSERT_ID();

    IF p_advance > 0 THEN
        INSERT INTO payments (rental_id, amount, payment_date, method)
        VALUES (v_rental_id, p_advance, p_rental_date, p_method);
    END IF;
    COMMIT;

    SELECT v_rental_id AS rental_id, p_rental_date AS rental_date, v_due AS due_date,
           p_days AS rental_days, v_rate AS daily_rate, v_total AS total_charge,
           p_advance AS advance_paid, v_total - p_advance AS balance_due;
END$$

```

Figure 6.1: Successful creation of the stored procedure

Testing

Successful rental: Customer 7 (Zain Abbas) rents vehicle 4 (ISB-3344, Suzuki Cultus at 3500 per day) for 3 days with a 5000 advance. Expected: total charge 10500, balance 5500, due date 2026-09-23, and the vehicle status changes to 'Rented'. Returning the vehicle then sets it back to 'Available' through the trigger.

Registers a new rental

```
1 CALL sp_register_rental(7, 4, '2026-09-20', 3, 5000.00, 'Cash');
```

Output

Extra options

rental_id	rental_date	due_date	rental_days	daily_rate	total_charge	advance_paid	balance_due
12	2026-09-20	2026-09-23	3	3500.00	10500.00	5000.00	5500.00

Figure 6.2: total charge 10500 and balance 5500

Failure cases (each must be rejected with an error message):

```
CALL sp_register_rental(8, 4, '2026-09-20', 2, 0, 'Cash'); -- vehicle 4 is rented again
CALL sp_register_rental(8, 5, '2026-09-20', 0, 0, 'Cash'); -- 0 days
CALL sp_register_rental(99, 5, '2026-09-20', 2, 0, 'Cash'); -- unknown customer
```

Case I

```
1 CALL sp_register_rental(8, 4, '2026-09-20', 2, 0, 'Cash');
```

Output

MySQL said: ?

```
#1644 - Rental rejected: vehicle is not available
                    (already rented or under maintenance).
```

Figure 6.4: Failure test: same vehicle rented again (trigger error)

Case II

```
1 CALL sp_register_rental(8, 5, '2026-09-20', 0, 0, 'Cash');
```

Output

MySQL said: ?

```
#1644 - Rental period must be at least 1 day.
```

Figure 6.5: Failure tests: 0 days and unknown customer (validation errors)

Case II

```
1 CALL sp_register_rental(99, 5, '2026-09-20', 2, 0, 'Cash');
```

Output

```
CALL sp_register_rental(99, 5, '2026-09-20', 2, 0, 'Cash');
```

MySQL said: ?

```
#1644 - Customer does not exist.
```

Figure 6.6: After the return update, vehicle 4 status = Available again

Return Car

```
1 UPDATE rentals SET return_date = '2026-09-23'
2 WHERE vehicle_id = 4 AND return_date IS NULL;
3
4 SELECT vehicle_number, status FROM vehicles WHERE vehicle_id = 4;
```

Output

	vehicle_number	status
☐  Edit  Copy  Delete	ISB-3344	Available

Task 7: Optimization Analysis

7.1 The problem

A typical management question is “which rentals were made in 2026?”. Written naturally, the query applies a function to the date column:

```
SELECT rental_id, customer_id, rental_date, total_charge
FROM rentals
WHERE YEAR(rental_date) = 2026;
```

In addition, `rentals` has no index on `rental_date`, although date filters are the most common way this table will be queried in reports.

7.2 Why it affects performance

- Applying `YEAR()` to the column makes the condition **non-sargable**: the database cannot search an index on `rental_date`, because the index stores dates, not years. It must read every row and compute `YEAR()` for each one.

- This is a full table scan, so the cost grows linearly with the table. With 11 sample rows this is invisible, but a real company would accumulate tens of thousands of rentals, and monthly or yearly reports would become slower every year.
- Adding an index alone does not fix it, because the function still hides the column from the index.

7.3 The improvement

Two changes are needed together: create an index on `rental_date`, and rewrite the condition as a date range on the bare column, so the index can be used for a range scan.

```
-- 1. Index on the filtered column
CREATE INDEX idx_rentals_rental_date ON rentals (rental_date);

-- 2. Sargable rewrite (same result, half-open range)
SELECT rental_id, customer_id, rental_date, total_charge
FROM rentals
WHERE rental_date >= '2026-01-01' AND rental_date < '2027-01-01';
```

The `EXPLAIN` statement was run at three points to compare the execution plans:

Step	Query form	What to look for in EXPLAIN
Before	<code>YEAR(rental_date) = 2026</code> , no index	type = ALL, key = NULL: full table scan
Index only	<code>YEAR(rental_date) = 2026</code> , index exists	Still a scan: the function prevents use of the index
After	Range on <code>rental_date</code> , index exists	possible_keys = idx_rentals_rental_date; type = range when the index is chosen

*Note: the sample table has only 11 rows, so the optimizer may still choose a full scan even when an index is available, because reading a very small table is cheaper than using the index. The important difference is that the rewritten query **can** use the index (possible_keys), whereas the `YEAR()` version never can. On a large table, the range scan reads only the matching rows.*

```
-- BEFORE : non-sargable query, no index on rental_date
EXPLAIN SELECT rental_id, customer_id, rental_date, total_charge
FROM rentals
WHERE YEAR(rental_date) = 2026;
```

Figure 7.1: EXPLAIN of the `YEAR(rental_date) = 2026` query before the index

```
-- Improvement 1 : index
CREATE INDEX idx_rentals_rental_date ON rentals (rental_date);
```

Figure 7.2: `CREATE INDEX idx_rentals_rental_date` executed successfully

```
-- Same result set, better plan
SELECT rental_id, customer_id, rental_date, total_charge
FROM rentals
WHERE rental_date >= '2026-01-01' AND rental_date < '2027-01-01';
```

Figure 7.3: EXPLAIN of the rewritten range query after the index

7.4 Data consistency safeguards in the design

Two columns hold derived values, which can be a consistency risk if they are updated by hand. The design protects them:

- `vehicles.status` is derived from the rentals; it is updated only by the triggers, and the BEFORE INSERT trigger blocks double rentals.
- `rentals.total_charge` is calculated in one place, the stored procedure (rate × days), and protected by CHECK constraints. The balance due is **not** stored anywhere; it is calculated by the view, so it can never become stale.

Conclusion

The CarGo Rentals database removes the redundancy and inconsistency of the spreadsheet approach by storing each fact once in a normalized (3NF) schema with proper keys and constraints. The JOIN queries and the view provide the reports management asked for, the triggers guarantee that a rented vehicle cannot be rented again, the stored procedure registers rentals with validation and automatic charge calculation, and the optimization analysis shows how a common report query can be made index-friend